Fiche de référence : commande `atelier` et API Python

Installer et lancer : `./scripts/installer.sh`, puis `source .venv/bin/activate` (ou préfixez par `.venv/bin/`). Tout est local. Options globales, **avant** la commande : `--rapide` (jeux réduits) · `--complet` (jeux complets) · `--secours` (décisions pré-calculées, sans modèle).

Commande	Ce qu'elle fait
`atelier info [--vitesse]`	moteur, mode, cache ; mesure la vitesse de la machine
`atelier decider "MESSAGE" [--options travail/options.toml] [--json]`	une décision : choix, marge, probabilités
`atelier evaluer [--options F] [--jeu dev] [--erreurs] [--acceptable] [--csv F]`	exactitude + IC 95 % + ECE sur un jeu étiqueté
`atelier seuils [--options F] [--pas 0.1]`	automatisation et erreurs pour chaque seuil de marge
`atelier accord [travail/annotations.csv]`	accord brut, κ de Cohen, κ de Fleiss, messages disputés
`atelier securite commande "CMD" [--tache T] [--config travail/securite.toml]`	garde-fou : autoriser / demander / refuser
`atelier securite message "TEXTE" [--expediteur E] [--canal SMS] [--signaux]`	P(frauduleux) et signaux oui / non
`atelier securite injection "TEXTE"` · `atelier securite evaluer`	injection (limite) · bilan garde + arnaques sur dev
`atelier missions` · `atelier preparer` · `atelier verifier N [--dossier travail]`	étapes 1 à 6, vérifiées **sur dev**
`ATELIER_ANIMATEUR=1 atelier verifier N --final` · `… evaluer --jeu test`	test scellé : **animateur seulement, une fois**

Fichiers des étapes (`travail/`) — `mission1.toml` · `cadrage.toml` · `annotations.csv` · `options.toml` (`question`, `[options]`, `[actions]`) · `politique.toml` (`seuil`, `erreurs_max_pour_100`) · `securite.toml` (`[garde] seuil`, `verifs` et `[arnaques] seuil`) ou `garde.py`.

wyat seul (0.1.2)

```
python import wyat m = wyat.load("teilomillet/wyat-1-800m", backend="mlx") # Mac ; ailleurs : device="cpu" ou "cuda" d = m.decide(wyat.Request.choice("Quel service ?", {"paiement": "Paiement, double débit", "livraison": "Colis"}, context=texte)) d.selected, d.probability_margin, d.probabilities_by_id # choix, marge, probabilités m.decide(wyat.Request.noul("La commande envoie-t-elle des données dehors ?", context={"commande": c})).probability_true m.decide(wyat.Request.score("Priorité ?", ["P4 …", "P3 …", "P2 …", "P1 …"], context=alerte)).expected_ordinal_value from wyat.evaluation import EvaluationCase, evaluate # rapport : accuracy, expected_calibration_error, r = evaluate(m, [EvaluationCase(id="1", request=req, expected=("paiement",))]) # selective(min_margin=0.5), calibrate()
```

En ligne de commande : `wyat decide teilomillet/wyat-1-800m --backend mlx < requetes.jsonl` · en service local : `wyat serve teilomillet/wyat-1-800m --backend mlx` puis `POST /v1/decide`.

Paquet `atelier` (`import atelier as at`)

Fonction	Rôle
`at.charger_modele()`	charge wyat (moteur choisi seul, cache disque, mode secours)
`at.decider(m, msg, options=at.OPTIONS)` · `at.probas(d)`	une décision · ses probabilités triées
`at.lire("dev" / "test")` · `at.lire_options("travail/options.toml")`	jeux étiquetés · options TOML
`at.evaluer(m, df, options)` → `(rapport, resultats)`	évaluation ligne par ligne
`at.intervalle_bootstrap(res.correct)` · `at.courbe_selective(rapport, seuils)`	IC 95 % · automatisation / erreurs
`at.router(m, msg, seuil=0.5)`	la politique : service, ou « humain »
`at.kappa_cohen(a, b)` · `at.kappa_fleiss(table)` · `at.accord_moyen(table)`	accord entre annotateurs
`at.garde(m, tache, cmd, reglage=at.ReglageGarde(0.4))`	garde-fou : `(action, raison, p)`
`at.p_arnaque(m, canal, expediteur, message)` · `at.signaux_arnaque(m, ligne)`	arnaques

Variables : `WYAT_MODEL` (copie locale) · `WYAT_BACKEND` · `WYAT_DEVICE` · `WYAT_MODE=auto|rapide|complet` · `WYAT_THREADS` · `WYAT_PRECALCULE=1` · `WYAT_CACHE=0` · `ATELIER_ANIMATEUR=1`.